

PhotoCryX

Installation of PhotoCryX using Anaconda

Anaconda or Miniconda is to be installed on the system, as we have done everything as a conda environment

[Download Anaconda](#)

After installing conda and activating it, in a terminal execute

```
conda create -n mp -c conda-forge pymeep
```

This will create a new environment "mp" which installs serial MEEP.

[Reference](#)

The installed conda environment can be activated by,

```
conda activate mp
```

Currently, MPB works only in serial mode (in Python); we do not require a parallel MEEP installation. We need some additional packages for the GUI, which are Tkinter and KLayout, for which the installation is straightforward. Tkinter is for the GUI, and KLayout is a GDSII file viewer and editor.

Tkinter can be installed using conda-forge or pip

```
conda install -c conda-forge tk
```

```
pip install tkinter
```

To install KLayout, download the "klayout*.deb" file and install

[Download KLayout](#)

Or it can be built from source

These are the prerequisites for PhotoCryX

To install PhotoCryX, download the latest version anywhere on your system, unzip it, and you are ready to compute. A desktop icon can be created using the launcher script provided in PhotoCryX. After the installation of the required packages and unzipping PhotoCryX (say in the "Downloads" folder), we will change the directory as,

```
cd ~/Downloads
```

```
git clone https://github.com/donmathew008/PhotoCryX
```

```
cd PhotoCryX
```

```
unzip PhotoCryX.zip
```

```
cd PhotoCryX
```

Then we will create a desktop launcher by executing "makeLauncher_PhotoCryX.sh" after making it executable by,

```
chmod +x makeLauncher_PhotoCryX.sh
```

```
./makeLauncher_PhotoCryX.sh
```

This will create "PhotoCryX.desktop", which can be copied to the desktop

```
cp PhotoCryX.desktop ~/Desktop
```

Right-click on the Desktop icon and give "Allow Launching"

Thus PhotoCryX can be launched directly from the desktop

After creating the launcher, right-click on the desktop icon and give "Allow Launching"

Now the installation is completed

For users who prefer direct execution mode via a terminal, the executables should be added to the current path. They are stored in the folder "EXEC" inside PhotoCryX, so to add to the path,

```
export PATH=~/.Downloads/PhotoCryX/PhotoCryX/EXEC:$PATH
```

Thus, we can directly use the executable in whichever parent folder contains the required inputs: "mpb.in" and the GDSII file.

PhotoCryX main Directory Structure

```

PhotoCryX/
├── code.py                                # Master code for the GUI
├── makeLauncher_PhotoCryX.sh             # Launcher script
├── photocryx.png                         # PhotoCryX icon
├── 1D/                                  # Master folder for 1D
│   └── band.py                          # Master code for 1D Photonic crystal
├── 2D/                                  # Master folder for 2D
│   ├── hexagonal/
│   │   └── band.py                      # Master code for 2D Hexagonal lattice
│   ├── honeycomb/
│   │   └── band.py                      # Master code for 2D Honeycomb lattice
│   └── square/
│       └── band.py                      # Master code for 2D Square lattice
├── 3D/                                  # Master folder for 3D
│   ├── BCC/
│   │   ├── band.py                    # Master code for 3D Body Centred Cubic
│   │   └── plot3d.py
│   ├── diamond/
│   │   ├── band.py                    # Master code for 3D Diamond lattice
│   │   └── plot3d.py
│   ├── FCC/
│   │   ├── band.py                    # Master code for 3D Face Centred Cubic
│   │   └── plot3d.py
│   └── SC
│       ├── band.py                    # Master code for 3D Simple Cubic (SC)
│       └── plot3d.py
├── EXEC/                                # Master EXEC folder with executables
│   ├── mpb_1d -> mpb_1d.py
│   ├── mpb_hx -> mpb_hx.py
│   ├── mpb_sq -> mpb_sq.py
│   └── mpb_vy -> mpb_vy.py
└── TEST/                                # Test Directory with "mpb.in" and code
    ├── 1d/
    ├── hex/
    ├── square/
    └── valley/

```

3D Visualization

The following steps are only needed for 3D visualization and can be skipped if not needed

Python directly doesn't include 3D visualization; thus, we need external packages (like Mayavi, Vis5d etc.)

Initially deactivate 'mp' if activated, as we don't wish to update any versions of the libraries needed for 'mp' environment

```
conda deactivate
```

Here, we will be installing all in the Downloads folder

- HDF5-1.14.6
- vis5d+-1.3.0-beta
- h5utils-1.13.1
- Mayavi

Update and build prerequisites

```
sudo apt update
```

```
sudo apt install -y build-essential gcc g++ gfortran make autoconf automake
```

```
cd ~/Downloads
```

HDF5-1.14.6

[Download HDF5-1.14.6](#)

```
unzip hdf5-1.14.6.zip
```

```
cd hdf5-1.14.6
```

```
./configure --enable-shared --enable-fortran --enable-parallel
```

```
make
```

```
sudo make install
```

The command below may be used in terminals while dealing with HDF5 files

```
export HDF5_USE_FILE_LOCKING=FALSE
```

vis5d+-1.3.0-beta

[Download vis5d+-1.3.0-beta](#)

```
tar -xvzf vis5d+-1.3.0-beta.tar.gz
```

```
cd vis5d+-1.3.0-beta
```

```
./configure --enable-threads --with-mesa --disable-shared --enable-static
```

Vis5D is an old package so we need some modifications in the source file to make it work in newer systems, by executing

```
sed -i 's/\
```

```
grep -Rnw --include='*.c' --include='*.h' '\<round\>|\<vis5d_round\>' src/
```

```
make clean
```

```
make
```

```
sudo make install
```

h5utils-1.13.1

[Download h5utils-1.13.1](#)

```
tar -xvzf h5utils-1.13.1.tar.gz
```

```
cd h5utils-1.13.1
```

```
./configure
```

```
make
```

```
sudo make install
```

Mayavi

[Reference](#)

```
pip install mayavi
```